MOCA_API_TESTS

MOCA ACS Proxy API - Pruebas con CURL (based on Genie)

Conjunto de pruebas para validar la API proxy de MOCA ACS expuesta en `http://IP_PRIVADA:3000/api/mocaacs`

Nota: Reemplazar `IP_PRIVADA` con la IP privada real (ej: 10.0.2.14)

Variables de Entorno (Configurar según tu entorno)

```bash
# Dirección del servidor MOCA ACS (IP privada)
export MOCA_IP="10.0.2.14"
export MOCA_PORT="3000"
export MOCA_URL="http://${MOCA_IP}:${MOCA_PORT}/api/mocaacs"

# ID de un dispositivo de prueba
export DEVICE_ID="MANUFACTURER-MODEL-SERIAL"
```

---

1. DISPOSITIVOS (Devices)

1.1 Listar todos los dispositivos

```bash
curl -i "${MOCA_URL}/devices"
```

1.2 Buscar dispositivo por ID

```bash
curl -i "${MOCA_URL}/devices" \
  --data-urlencode 'query={"_id":"'"${DEVICE_ID}"'"}'
```

1.3 Buscar dispositivos online (últimos 5 minutos)

```bash
curl -i "${MOCA_URL}/devices" \
  --data-urlencode 'query={"_lastInform":{"$gt":"'"$(date -d '5 minutes ago' '+%Y-%m-%d %H:%M:%S +0000')"'"}}'
```

1.4 Buscar por dirección MAC

```
curl -i "${MOCA_URL}/devices" \
  --data-urlencode 'query=
{"InternetGatewayDevice.WANDevice.1.WANConnectionDevice.1.WANIPConnection.
1.MACAddress":"20:2B:C1:E0:06:65"}'
```

1.5 Obtener dispositivo específico

```
curl -i "${MOCA_URL}/devices/${DEVICE_ID}"
```

1.6 Obtener proyección específica (parámetros selectivos)

```
curl -i "${MOCA_URL}/devices" \
  --data-urlencode 'query={"_id":"'"${DEVICE_ID}"'"}' \
  --data-urlencode
'projection=InternetGatewayDevice.DeviceInfo.ModelName,InternetGatewayDevi
ce.DeviceInfo.Manufacturer,InternetGatewayDevice.DeviceInfo.SoftwareVersio
n'
```

1.7 Eliminar dispositivo

```
curl -i -X DELETE "${MOCA_URL}/devices/${DEVICE_ID}"
```

---

2. TAREAS (Tasks)

2.1 Listar todas las tareas

```
curl -i "${MOCA_URL}/tasks"
```

2.2 Listar tareas de un dispositivo específico

```
curl -i "${MOCA_URL}/tasks" \
  --data-urlencode 'query={"device":"'"${DEVICE_ID}"'"}'
```

2.3 Crear tarea - Refresh de parámetros

```
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tasks?
connection_request" \
  -H "Content-Type: application/json" \
  -d '{"name":"refreshObject","objectName":""}'
```

2.4 Crear tarea - Cambiar parámetro (SSID WiFi)

```
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tasks?
connection_request" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "setParameterValues",
    "parameterValues": [
      ["InternetGatewayDevice.LANDevice.1.WLANConfiguration.1.SSID",
"MiRedWiFi", "xsd:string"]
    ]
  }'
```

2.5 Crear tarea - Cambiar contraseña WiFi

```
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tasks?
connection_request" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "setParameterValues",
    "parameterValues": [

["InternetGatewayDevice.LANDevice.1.WLANConfiguration.1.PreSharedKey.1.Pre
SharedKey", "NuevaPassword123", "xsd:string"]
    ]
  }'
```

2.6 Crear tarea - Reiniciar dispositivo

```
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tasks?
connection_request" \
  -H "Content-Type: application/json" \
  -d '{"name":"reboot"}'
```

2.7 Crear tarea - Sin connection request (ejecutar en próximo inform)

```
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tasks" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "setParameterValues",
    "parameterValues": [
      ["InternetGatewayDevice.ManagementServer.PeriodicInformEnable",
"true", "xsd:boolean"],
      ["InternetGatewayDevice.ManagementServer.PeriodicInformInterval",
"300", "xsd:int"]
```

```
    ]
  }'
```

2.8 Reintentar tarea fallida

```
export TASK_ID="5403908ef28ea3a25c138adc"
curl -i -X POST "${MOCA_URL}/tasks/${TASK_ID}/retry"
```

2.9 Eliminar tarea

```
export TASK_ID="5403908ef28ea3a25c138adc"
curl -i -X DELETE "${MOCA_URL}/tasks/${TASK_ID}"
```

---

3. FALLOS (Faults)

3.1 Listar todos los fallos

```
curl -i "${MOCA_URL}/faults"
```

3.2 Listar fallos de un dispositivo

```
curl -i "${MOCA_URL}/faults" \
  --data-urlencode 'query={"_id":"'"${DEVICE_ID}"':*"}'
```

3.3 Eliminar fallo

```
export FAULT_ID="${DEVICE_ID}:default"
curl -i -X DELETE "${MOCA_URL}/faults/${FAULT_ID}"
```

---

4. TAGS

4.1 Agregar tag a dispositivo

```
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tags/testing"
```

4.2 Agregar múltiples tags

```
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tags/production"
curl -i -X POST "${MOCA_URL}/devices/${DEVICE_ID}/tags/priority"
```

4.3 Remover tag

```
curl -i -X DELETE "${MOCA_URL}/devices/${DEVICE_ID}/tags/testing"
```

---

5. PRESETS

5.1 Listar todos los presets

```
curl -i "${MOCA_URL}/presets"
```

5.2 Crear preset - Configurar intervalo de inform

```
curl -i -X PUT "${MOCA_URL}/presets/inform-interval" \
  -H "Content-Type: application/json" \
  -d '{
    "weight": 100,
    "precondition": "{\"_tags\": \"test\"}",
    "configurations": [
      {
        "type": "value",
        "name":
"InternetGatewayDevice.ManagementServer.PeriodicInformEnable",
        "value": "true"
      },
      {
        "type": "value",
        "name":
"InternetGatewayDevice.ManagementServer.PeriodicInformInterval",
        "value": "300"
      }
    ]
  }'
```

5.3 Crear preset - Desabilitar WiFi

```
curl -i -X PUT "${MOCA_URL}/presets/disable-wifi" \
  -H "Content-Type: application/json" \
  -d '{
    "weight": 50,
    "precondition": "{}",
```

```
    "configurations": [
      {
        "type": "value",
        "name":
"InternetGatewayDevice.LANDevice.1.WLANConfiguration.1.Enable",
        "value": "false"
      }
    ]
  }'
```

5.4 Eliminar preset

```
curl -i -X DELETE "${MOCA_URL}/presets/inform-interval"
```

---

6. ARCHIVOS (Files)

6.1 Listar todos los archivos

```
curl -i "${MOCA_URL}/files"
```

6.2 Subir archivo firmware

```
curl -i -X PUT "${MOCA_URL}/files/firmware-v1.0.bin" \
  -H "fileType: 1 Firmware Upgrade Image" \
  --data-binary @/path/to/firmware.bin
```

6.3 Subir archivo de configuración

```
curl -i -X PUT "${MOCA_URL}/files/config.xml" \
  -H "fileType: 3 Vendor Configuration File" \
  --data-binary @/path/to/config.xml
```

SCRIPT DE PRUEBA AUTOMATIZADO

Crear archivo `test-mocaacs-api.sh`:

```
#!/bin/bash

set -e
```

```bash
# Colores para output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

# Variables
export MOCA_IP="${1:-10.0.2.14}"
export MOCA_PORT="3000"
export MOCA_URL="http://${MOCA_IP}:${MOCA_PORT}/api/mocaacs"

echo -e "${YELLOW}=== Pruebas API mocaACS Proxy ===${NC}"
echo "URL: ${MOCA_URL}"
echo ""

# Test 1: Health check (desde la ruta principal)
echo -e "${YELLOW}Test 1: Health Check${NC}"
if curl -s "http://${MOCA_IP}:${MOCA_PORT}/api/health" | grep -q "ok";
then
  echo -e "${GREEN}✓ Server está activo${NC}"
else
  echo -e "${RED}✗ Server no responde${NC}"
  exit 1
fi
echo ""

# Test 2: Listar dispositivos
echo -e "${YELLOW}Test 2: Listar dispositivos${NC}"
DEVICES=$(curl -s "${MOCA_URL}/devices")
DEVICE_COUNT=$(echo "$DEVICES" | grep -o '_id' | wc -l)
echo -e "${GREEN}✓ ${DEVICE_COUNT} dispositivos encontrados${NC}"
echo ""

# Test 3: Listar tareas
echo -e "${YELLOW}Test 3: Listar tareas${NC}"
TASKS=$(curl -s "${MOCA_URL}/tasks")
TASK_COUNT=$(echo "$TASKS" | grep -o '"_id"' | wc -l)
echo -e "${GREEN}✓ ${TASK_COUNT} tareas encontradas${NC}"
echo ""

# Test 4: Listar fallos
echo -e "${YELLOW}Test 4: Listar fallos${NC}"
FAULTS=$(curl -s "${MOCA_URL}/faults")
FAULT_COUNT=$(echo "$FAULTS" | grep -o '"_id"' | wc -l)
echo -e "${GREEN}✓ ${FAULT_COUNT} fallos encontrados${NC}"
echo ""

echo -e "${GREEN}=== Todas las pruebas pasaron ===${NC}"
```

Ejecutar:

```bash
chmod +x test-mocaacs-api.sh
./test-mocaacs-api.sh 10.0.2.14
```

Restricciones y Notas

1. **IP Privada:** Solo accesible desde IPs privadas (10.x.x.x, 172.16-31.x.x, 192.168.x.x)
2. **Sin autenticación JWT:** No requiere token JWT
3. **Rate limiting:** Implementar según necesidad
4. **Encodificación:** Los query parameters se envían URL-encoded
5. **Device IDs:** Deben estar correctamente escapados en URLs

---

Solución de Problemas

Error 403 - Acceso denegado

```
Solución: Asegúrate de ejecutar curl desde una IP privada
```

Error de conexión

```bash
# Verificar conectividad
ping 10.0.2.14
curl -i http://10.0.2.14:3000/api/health
```

Query no retorna resultados

```bash
# Verificar query JSON correcta
echo '{"_id":"DEVICE-ID"}' | jq .
```

Última actualización: 2026-04-20